



[BIT703 Web Technologies](#) / [Contents](#) / Topic 6: Client-side web application development – CSS / Understanding CSS

Grid layout 45m

Grid layout is another CSS page layout that we briefly mentioned earlier when we introduced all the page layout functions in CSS.



If you need a refresher about this, we recommend taking a look at the following pages:

[CSS page layout](#) →

The grid layout is defined by the grid property (`display: grid`). This defines a grid layout using rows and columns, making it easier to design a web page without the need to use floats or positioning (return to the page CSS page layout linked above if you don't remember what floats or positioning are regarding layout).

A grid consists of a container that has one or more child elements.

The gaps between the rows and columns can be adjusted using either:

- `column-gap`: – adjusts the gap between columns
- `row-gap`: – adjusts the gap between rows
- `gap`: – shorthand for the row-gap and column-gap property.

The main difference between the flexbox and the grid layout approaches is:

- Flexbox was designed with **one dimension** in mind, that is, you either focus on the row, or focus on the column.

[Skip to main content](#)

- Grid is a **two-dimension** layout that focuses on the rows and the columns at the same time.

For more on their differences, read *Relationship of Grid Layout to Other Layout Methods* (MDN, 2025)^{fnref:f4df26}.

Gap property

The gap property is used as shorthand for the row and column properties. It is written as:



Copy

```
gap: [row value] [column value];
```



For example, if you wanted your rows to have a gap of 50 px and your columns to have a gap of 100 px, you'd write the gap property as:



[Skip to main content](#)



gap: 50px 100px;



View our CodePen example below, where we've used the grid property to create a Tic-Tac-Toe game board (3×3 grid). Our code includes a grid container, which is the game board, and nine items, which are the X or O items.

Have a go at adjusting the gaps between the rows and columns.



[Skip to main content](#)

HTML

CSS

Result

EDIT ON

LIVE

```
.game-board {  
  display: grid;  
  grid-template-columns: auto auto  
auto;  
  background-color: #008080;  
  width: 400px;  
  height: 400px;  
  margin: 0 auto;  
  border: 6px solid #2c3e50;  
}  
.mark {  
  color: white;  
  text-align: center;  
  background-color: #008080;  
  font-family: Arial, Helvetica,  
sans-serif;  
  font-size: 72px;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  border: 6px solid #2c3e50;  
}
```

Resources1x0.5x0.25xRerun

Example 6.17 Tic-Tac-Toe



☒ Chessboard

Non-Assessed

[Skip to main content](#)

In this activity, have a go at creating a chessboard layout using the grid, grid-template-columns and grid-template-rows properties.

Remember, a chessboard consists of:

- ▶ 64 squares
- ▶ 8 rows and 8 columns
- ▶ alternating black and white squares as follows:

Screenshot © codepen.io

☐ I have successfully completed this task!

Submit



Dig deeper

If you want to learn more about the grid property, then read these resources:

- ▶ *Grid*, which provides a list of the grid values available in CSS as well as examples.
- ▶ *CSS Grid Layout Module*, which is a tutorial on grid lines, grid containers and grid items.



[Skip to main content](#)

Now, let's delve into CSS responsive web design, a technique for creating websites that adapt seamlessly to different screen sizes and devices.



Thoughts on this page?

